

HASSAN II UNIVERSITY  
FACULTY OF SCIENCES BEN M'SICK  
MATHEMATICS AND COMPUTER SCIENCE DEPARTMENT  
- CASABLANCA -

MASTER DATA SCIENCE AND BIG DATA (DSBD)  
END OF MODULE PROJECT

---

# Cartographie Sémantique et Analyse des Publications de la FSBM par NLP & Web Scraping

---

**Student:**  
Achraf CHAHID

**Supervisor:**  
Prof. Habib BEN LAHMAR

# Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Data Engineering &amp; Pipeline Architecture</b>        | <b>2</b>  |
| 1.1      | Overview . . . . .                                         | 2         |
| 1.2      | Robust Data Extraction (Bronze Layer) . . . . .            | 2         |
| 1.3      | OpenAlex Metadata Enrichment . . . . .                     | 4         |
| 1.4      | Automated PDF Acquisition via Selenium . . . . .           | 4         |
| 1.5      | Data Cleaning and Structuring (Silver Layer) . . . . .     | 5         |
| <b>2</b> | <b>Semantic Search &amp; NLP Integration</b>               | <b>6</b>  |
| 2.1      | Vector Embeddings . . . . .                                | 6         |
| 2.2      | Vector Database (ChromaDB) . . . . .                       | 6         |
| <b>3</b> | <b>System Architecture &amp; Web Application</b>           | <b>8</b>  |
| 3.1      | System Architecture Overview . . . . .                     | 8         |
| 3.2      | Backend Infrastructure (FastAPI) . . . . .                 | 8         |
| 3.2.1    | Core API Endpoints . . . . .                               | 8         |
| 3.3      | Frontend Interface & Application Views (Next.js) . . . . . | 9         |
| 3.3.1    | Semantic Search & Discovery Engine . . . . .               | 9         |
| 3.3.2    | Faculty Researcher Directory . . . . .                     | 10        |
| 3.3.3    | Individual Researcher Profile . . . . .                    | 11        |
| <b>4</b> | <b>Conclusion &amp; Perspectives</b>                       | <b>12</b> |
| 4.1      | Summary of Achievements . . . . .                          | 12        |
| 4.2      | Future Work . . . . .                                      | 12        |

# Chapter 1

## Data Engineering & Pipeline Architecture

### 1.1 Overview

The data engineering phase forms the critical foundation of the semantic search engine. To ensure data integrity, reproducibility, and high performance, the pipeline was designed following the **Medallion Architecture** (Bronze, Silver, Gold). This multi-stage approach guarantees that the system is idempotent, resilient to network interruptions, and respectful of external server rate limits while transforming unstructured web data into structured analytical datasets.

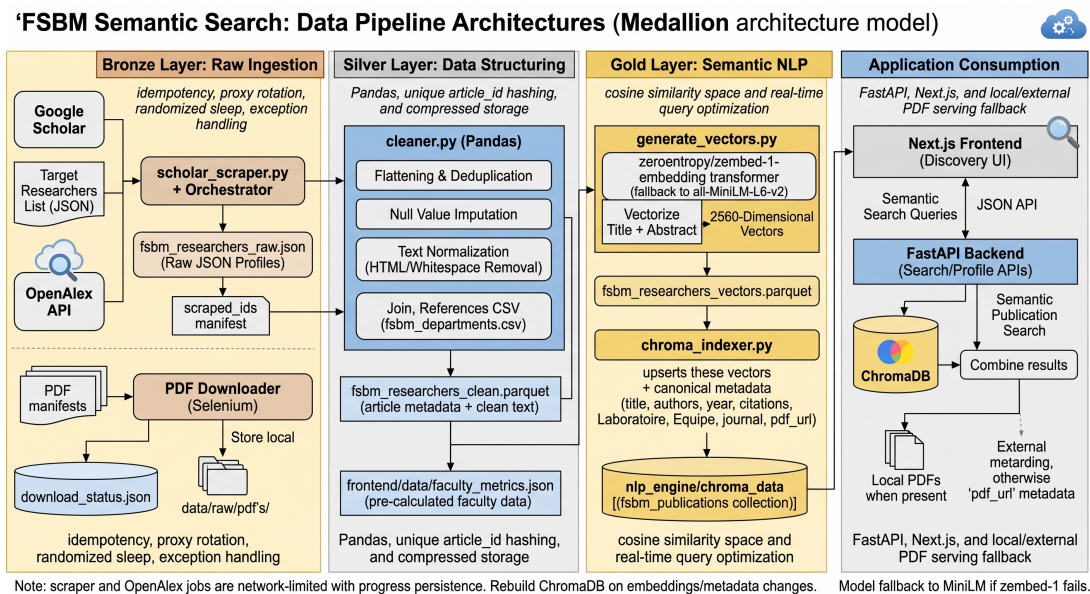


Figure 1.1: Overview of the Medallion Data Pipeline (Bronze to Gold layers)

### 1.2 Robust Data Extraction (Bronze Layer)

The initial phase involves extracting raw publication data using the `scholarly` Python library. Given the strict anti-bot mechanisms (such as CAPTCHAs and HTTP 429 Too Many Requests errors) deployed by Google Scholar, the scraper (`scholar_scraper.py`) and its orchestrator were engineered with high resilience:

- **Idempotency & Crash Recovery:** An atomic JSON-based manifest system (`scraped_ids` set) tracks successful extractions. If the script crashes or is manually halted, it seamlessly resumes exactly where it left off, preventing duplicate work and data corruption.

- **Rate Limiting & Politeness:** Randomized sleep intervals (`random.uniform(5.0, 10.0)`) are injected between profile requests to simulate human pacing and avoid triggering heuristic security triggers.
- **Proxy Rotation:** The integration of `ProxyGenerator` distributes requests across free proxy pools, effectively hiding the host IP address and bypassing IP-based temporary bans.
- **Exception Handling:** Comprehensive `try/except` blocks wrap the network calls, ensuring that a failure on a single publication or profile does not halt the entire pipeline.

```

Terminal Local Local (2) Local (3) Local (4) + -
[*.venv] PS C:\Users\chahid\PycharmProjects\Django_Project\fsbm-semantic-search> python data_pipeline/src/orchestrator.py
[*] Fetching and configuring free proxies... (This may take a minute)
[*] Proxy configured successfully! Hiding real IP.
[*] Loaded 95 target profiles.
[*] Found 74 profiles already scraped. Resuming...

[*] Processing Profile 1/95: Habib BEN LAHMAR
[*] Skipping Habib BEN LAHMAR - Already successfully scraped.

[*] Processing Profile 2/95: Nawal Sael
[*] Skipping Nawal Sael - Already successfully scraped.

[*] Processing Profile 3/95: Said Nough
[*] Skipping Said Nough - Already successfully scraped.

[*] Processing Profile 4/95: KHALID KANDALI
[*] Skipping KHALID KANDALI - Already successfully scraped.

[*] Processing Profile 5/95: faouzia Benabbou
[*] Skipping faouzia Benabbou - Already successfully scraped.

[*] Processing Profile 6/95: Mohamed Talea
[*] Skipping Mohamed Talea - Already successfully scraped.

[*] Processing Profile 76/95: SAID GMOUH
[*] Skipping SAID GMOUH - Already successfully scraped.

[*] Processing Profile 77/95: SIHAM SLASSI
[*] Skipping SIHAM SLASSI - Already successfully scraped.

[*] Processing Profile 78/95: SOUMIA MORDANE
[*] Fetching Google Scholar profile directly for ID: hINOScAAAAAJ
[*] Found profile: Soumia Mordane. Fetching detailed metrics...
[*] Fetching top 15 articles (with rate limiting)...
  -> [1/15] Fetching article: An improved coastal upwelling index from sea surface temperature using satellite-based approach-The case of the Canary Current upwelling system
  -> [2/15] Fetching article: Offshore wind energy resource in the Kingdom of Morocco: Assessment of the seasonal potential variability based on satellite data
  -> [3/15] Fetching article: Impact of the variational assimilation of ground-based GNSS zenith total delay into AROME-Morocco model
  -> [4/15] Fetching article: Radar rainfall estimation in Morocco: Quality control and gauge adjustment
  -> [5/15] Fetching article: Impact des vents sur l'upwelling au sud du Maroc; apport du modèle ROMS forcé par les données ALADIN et QuikSCAT
  -> [6/15] Fetching article: A parabolic equation based on a rational quadratic approximation for surface gravity wave propagation
  -> [7/15] Fetching article: Optical and thermo-plasmonic properties of spherical and hollow gold nanoparticles injected in cell organelles
  -> [8/15] Fetching article: Numerical analysis of surface plasmon resonance and photothermal properties of silica/gold/graphene and hollow/gold/graphene core-multi-shell nanoparticles
  -> [9/15] Fetching article: Wave energy potential analysis in the Casablanca-Mohammedia coastal area (Morocco)
  -> [10/15] Fetching article: Optical and thermoplasmonic properties of core (AuAgI-x)-shell (Au) nanostructures
  -> [11/15] Fetching article: Computation of nonlinear free-surface flows using the method of fundamental solutions
  -> [12/15] Fetching article: Improving heavy rainfall forecasts by assimilating surface precipitation in the convective scale model AROME: A case study of the Mediterranean event of November 4, 201
7
  -> [13/15] Fetching article: Wave reflection by rectangular breakwaters for coastal protection
  -> [14/15] Fetching article: Numerical modeling of the wave-plate-current interaction by the boundary element method
  -> [15/15] Fetching article: New numerical investigation using meshless methods applied to the linear free surface water waves
[*] Saved data for SOUMIA MORDANE to raw/ folder.
[*] Sleeping for 5.54 seconds before next profile...

[*] Processing Profile 79/95: ABDELWJID ABOURRICHE
[*] Fetching Google Scholar profile directly for ID: iKIUSQIAAAAJ

```

Figure 1.2: Terminal output demonstrating idempotent scraping, rate limiting, and automated exception handling during data extraction.

## 1.3 OpenAlex Metadata Enrichment

To bypass the limitations of Google Scholar's closed ecosystem, the pipeline utilizes the OpenAlex API. This stage cross-references the extracted titles to enrich the dataset with:

- Open Access (OA) PDF URLs for direct document retrieval.
- Exhaustive citation graphs (`referenced_works`) to understand paper relationships.
- Standardized unique identifiers (DOIs, OpenAlex IDs).

This enrichment process also utilizes an incremental saving mechanism, writing to `openalex_enrichment.json` after every successful HTTP 200 response to safeguard against API timeouts.

## 1.4 Automated PDF Acquisition via Selenium

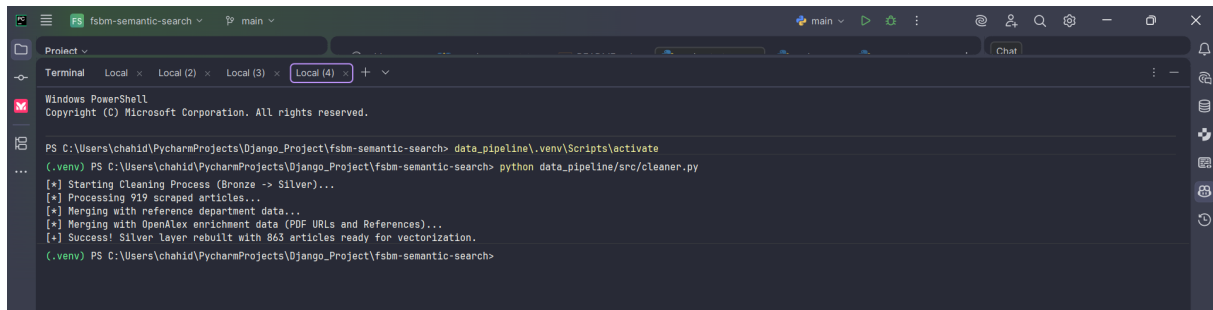
A significant challenge in academic data extraction is publisher Content Delivery Networks (CDNs, e.g., Elsevier, ScienceDirect) returning HTTP 403 Forbidden errors to standard programmatic requests (`requests` library).

To solve this, we implemented a headless automated browser using `Selenium` and `webdriver-manager`. By forcing Chrome to download PDFs directly via the `plugins.always_open_pdf_externally` preference, the pipeline naturally manages JavaScript execution and dynamic security tokens. A local state manifest (`download_status.json`) is maintained to ensure the idempotency of the downloads.

## 1.5 Data Cleaning and Structuring (Silver Layer)

The raw data extracted (Bronze layer) is highly nested and contains noise. The `cleaner.py` script utilizes the **Pandas** library to strictly structure and sanitize the dataset, ensuring it is primed for Machine Learning integration:

- **Flattening & Deduplication:** The nested JSON profiles are flattened into a strict tabular format. Duplicate publications (often caused by co-authorship within the same faculty) are identified and removed using unique `article_id` hashing.
- **Null Value Imputation:** Missing metadata (e.g., missing journals, empty citation arrays) are properly coerced into native Python `None` or empty lists, preventing generic string artifacts from polluting the NLP vectorization phase.
- **Text Normalization:** Publication abstracts undergo basic Natural Language Processing (NLP) pre-treatment: removal of residual HTML tags, conversion to lower-case, and elimination of erratic whitespace (`abstract_clean`).
- **Optimized Storage (Parquet):** The final cleaned dataset is exported as `fsbm_researchers_clean.parquet`. This columnar storage format drastically reduces file size through compression and significantly accelerates I/O operations when loading data into RAM for the GPU-intensive vectorization step.



```

PS C:\Users\chahid\PycharmProjects\Ojango_Project\fsbm-semantic-search> data_pipeline\.venv\Scripts\activate
(.venv) PS C:\Users\chahid\PycharmProjects\Ojango_Project\fsbm-semantic-search> python data_pipeline/src/cleaner.py
[*] Starting Cleaning Process (Bronze -> Silver)...
[*] Processing 919 scraped articles...
[*] Merging with reference department data...
[*] Merging with OpenAlex enrichment data (PDF URLs and References)...
[*] Success! Silver layer rebuilt with 863 articles ready for vectorization.
(.venv) PS C:\Users\chahid\PycharmProjects\Ojango_Project\fsbm-semantic-search>

```

Figure 1.3: Terminal output demonstrating the Pandas data structuring process, including deduplication, text normalization, and Parquet export.

# Chapter 2

## Semantic Search & NLP Integration

### 2.1 Vector Embeddings

Traditional keyword-based search is insufficient for academic literature, where concepts can be expressed using diverse terminology. To enable semantic search, we transform textual data (abstracts, titles, and extracted PDF text) into dense vector representations. The system utilizes the **zembed-1** model hosted via HuggingFace, providing high-dimensional contextual embeddings.

### 2.2 Vector Database (ChromaDB)

The generated embeddings are stored and indexed using **ChromaDB**. ChromaDB was selected for its efficiency in handling high-dimensional vector similarity searches. When a user submits a query, it is embedded using the same NLP model, and ChromaDB performs a K-Nearest Neighbors (KNN) search using cosine similarity to return the most contextually relevant research papers.

```

fsbm-semantic-search  add/ldempotency
Project: fsbm-semantic-search
Terminal: Local x Local (2) x +
[+] sentence_bert_config.json: 100% | 55.0/55.0 [00:00:00:00, 150kB/s]
[+] config.json: 100% | 1.56k/1.56k [00:00:00:00, 5.84kB/s]
[+] model.safetensors.index.json: 100% | 32.9k/32.9k [00:00:00:00, 61.3kB/s]
[+] Fetching 2 files: 100% | 2/2 [1:42:50:00:00, 3085.45s/it]
[+] Download complete: 100% | 7.01GB, 1.01MB/s
[+] Reconstruction complete: 100% | 8.04GB / 8.04GB, 1.22MB/s
[+] Loading weights: 100% | 398/398 [00:14:00:00, 26.89it/s]
[+] tokenizer_config.json: 100% | 5.43k/5.43k [00:00:00:00, 1.99MB/s]
[+] vocab.json: 100% | 2.78M/2.78M [00:01:00:00, 1.47MB/s]
[+] merges.txt: 100% | 1.67M/1.67M [00:00:00:00, 2.25MB/s]
[+] tokenizer.json: downloading bytes: | 3.40MB, 270kB/s
[+] tokenizer.json: reconstructing file: 100% | 11.4MB / 11.4MB, 973kB/s
[+] added_tokens.json: 100% | 707/707 [00:00:00:00, 1.21MB/s]
[+] special_tokens_map.json: 100% | 613/613 [00:00:00:00, 2.34MB/s]
[+] chat_template.jinja: 100% | 4.17k/4.17k [00:00:00:00, 5.34MB/s]
[+] config.json: 100% | 297/297 [00:00:00:00, 615kB/s]
Default prompt name is set to 'document'. This prompt will be applied to all inference calls, except if a 'prompt' or 'prompt_name' parameter is provided.
[+] Encoding 359 abstracts... (This may take a minute depending on your CPU/GPU)
Batches: 100% | 12/12 [3:32:31:00:00, 1862.62s/it]
[+] Success! All abstracts vectorized.
[+] Vector Dimension: 2560
[+] Gold Layer saved to: C:\Users\chahid\PycharmProjects\Django_Project\fsbm-semantic-search\data_pipeline\data\processed\fsbm_researchers_vectors.parquet
(.venv) PS C:\Users\chahid\PycharmProjects\Django_Project\fsbm-semantic-search>
(.venv) PS C:\Users\chahid\PycharmProjects\Django_Project\fsbm-semantic-search>
(.venv) PS C:\Users\chahid\PycharmProjects\Django_Project\fsbm-semantic-search> python nlp_engine/src/chroma_indexer.py
[+] Starting ChromaDB Indexing Process...
[+] Loaded 359 vectorized articles.
[+] Connected to ChromaDB Collection: 'fsbm_publications'
[+] Inserting 359 documents into ChromaDB...
100% | 1/1 [00:00:00:00, 3.08it/s]
[+] Success! All vectors indexed in ChromaDB.
[+] Database saved locally at: C:\Users\chahid\PycharmProjects\Django_Project\fsbm-semantic-search\nlp_engine\chroma_data
(.venv) PS C:\Users\chahid\PycharmProjects\Django_Project\fsbm-semantic-search>
fsbm-semantic-search  nlp_engine  src  chroma_indexer.py
62.50 CRLF UTF-8 4 spaces Python 3.13

```

Figure 2.1: Terminal output demonstrating the vectorization of textual data and the upsertion of dense embeddings into the ChromaDB vector store.

Figure 2.1 illustrates the execution log from the initial baseline run, where 359 article abstracts were successfully encoded into 2560-dimensional dense vectors using the **zeroentropy/zembed-1-embedding** transformer model and indexed into the local ChromaDB store. Following the complete faculty scraping and data cleaning phases, the corpus

has scaled to over 900 publication records (with approximately 850 valid abstracts after removing empty entries). Because generating high-dimensional embeddings across a corpus of this scale requires substantial compute, batch inference is being executed using an external cloud acceleration environment (Google Colab) to bypass local CPU memory bottlenecks before writing the final vectors to the Gold Parquet layer and rebuilding the vector index.

---

# Chapter 3

## System Architecture & Web Application

---

### 3.1 System Architecture Overview

The semantic discovery platform is designed as a decoupled, micro-service oriented web application. The core system architecture separates compute-intensive vector operations from presentation concerns, facilitating independent scalability, modular maintenance, and low-latency interaction.

- **Data & Vector Engine:** ChromaDB handles vector persistence and approximate nearest neighbor (ANN) retrieval using cosine distance spaces, underpinned by embeddings pre-computed via the `zeroentropy/zembed-1-embedding` transformer model.
- **High-Performance API Backend:** Implemented with **FastAPI**, orchestrating semantic queries, metadata extraction, dynamic similarity score normalization, and PDF document serving.
- **Interactive User Interface:** A reactive, modern Single-Page Application (SPA) built with **Next.js** (React) and styled with **Tailwind CSS**, delivering interactive filtering, real-time query rendering, and responsive academic metrics visualization.

---

### 3.2 Backend Infrastructure (FastAPI)

The backend acts as the bridge between the persisted data lakehouse (Parquet and ChromaDB) and the client application. FastAPI was chosen due to its asynchronous ASGI architecture, automatic OpenAPI/Swagger documentation, and native Pydantic data validation.

#### 3.2.1 Core API Endpoints

The backend exposes three principal RESTful routes:

1. **GET /search:** Accepts a natural language query string alongside optional faceted filtering parameters (e.g., publication year, author ID). It dynamically encodes the prompt into dense vector space, queries ChromaDB for the top- $k$  nearest neighbors, and normalizes cosine distance ( $d$ ) into human-interpretable percentage match confidence ( $S = \max(0, (1 - d)) \times 100\%$ ).

2. **GET /faculty-profiles:** Returns aggregated faculty metadata, research laboratory affiliations, and cumulative bibliometric indicators (*h*-index, *i10*-index, citation counts).
3. **GET /faculty-profiles/{id}:** Retrieves deep-dive publication records, historical output, and OpenAlex enrichment identifiers for an individual academic profile.

### 3.3 Frontend Interface & Application Views (Next.js)

The frontend is architected using the Next.js App Router, leveraging client-side state management and server-rendered layout performance. The interface is organized around three primary user journeys.

#### 3.3.1 Semantic Search & Discovery Engine

The primary discovery interface provides an intuitive search bar allowing researchers and academic reviewers to query the scientific corpus conceptually rather than relying on exact keyword matching.

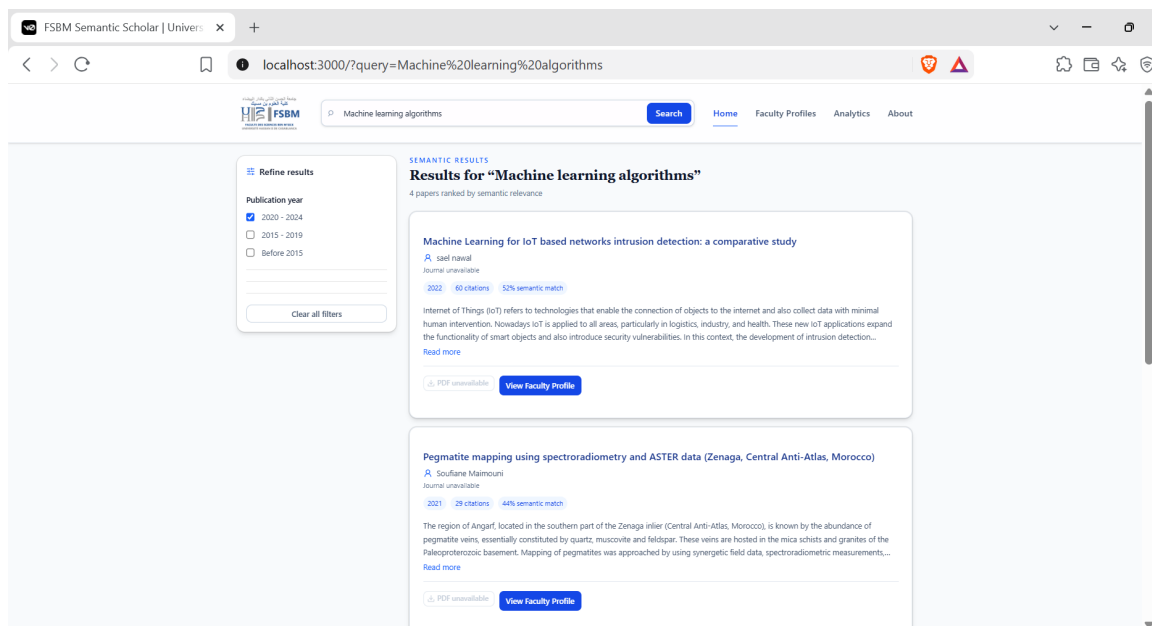


Figure 3.1: Semantic search interface demonstrating multi-faceted filtering by publication year, semantic confidence score badges, and paper metadata cards.

As shown in Figure 3.1, the search results view incorporates:

- **Faceted Navigation:** Dynamic sidebar filters allowing queries to be restricted by publication cohorts (e.g., 2020–2024, 2015–2019, or prior to 2015).
- **Semantic Match Badges:** Dynamic visual indicators showing normalized semantic relevance (e.g., 52% match) alongside total citation counts.
- **Direct Document Access:** Instant status indicators showing Open Access PDF availability and contextual links directly into the respective researcher's profile.

### 3.3.2 Faculty Researcher Directory

To allow exploratory browsing across departments and laboratories, the application provides a centralized institutional directory.

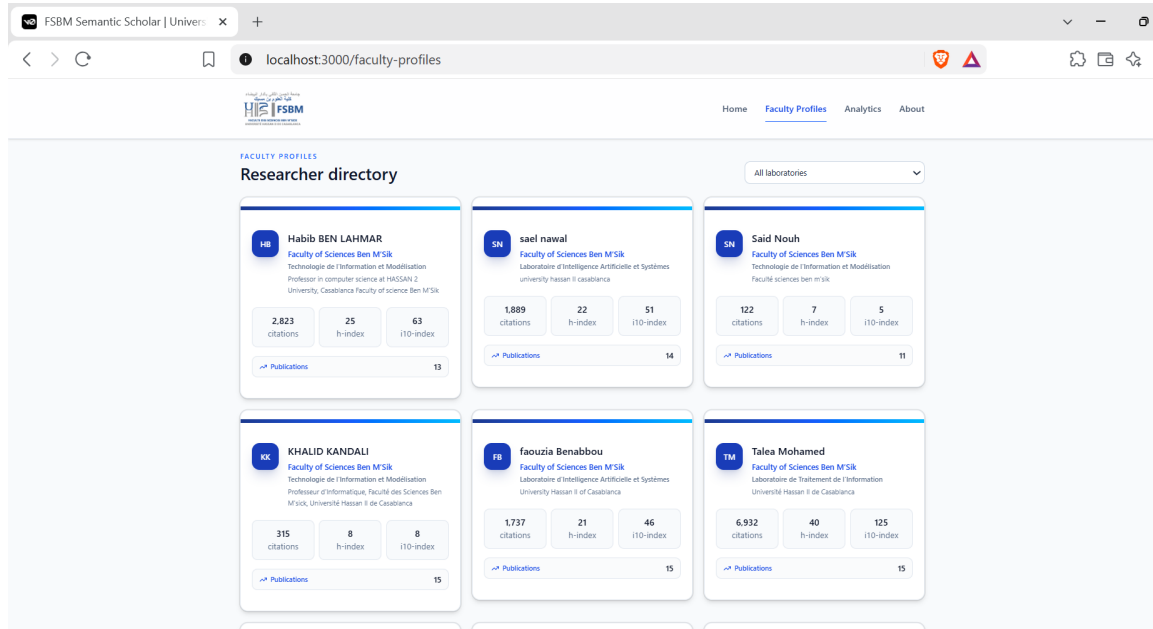


Figure 3.2: Faculty Researcher Directory displaying researcher affiliations, laboratory filters, and computed academic impact metrics ( $h$ -index,  $i10$ -index, total citations).

As highlighted in Figure 3.2, each faculty member is presented as a modular summary card featuring:

- **Institutional Affiliation & Laboratory:** Affiliation badges linked to laboratory groupings (e.g., *Laboratoire d'Intelligence Artificielle et Systèmes*).
- **Bibliometric Scorecards:** Standardized display of key academic indicators including total career citations,  $h$ -index, and  $i10$ -index.
- **Indexed Publication Count:** Exact number of cleansed, indexed publications available for downstream semantic matching.

### 3.3.3 Individual Researcher Profile

Clicking into a specific faculty profile navigates to an exhaustive academic overview view.

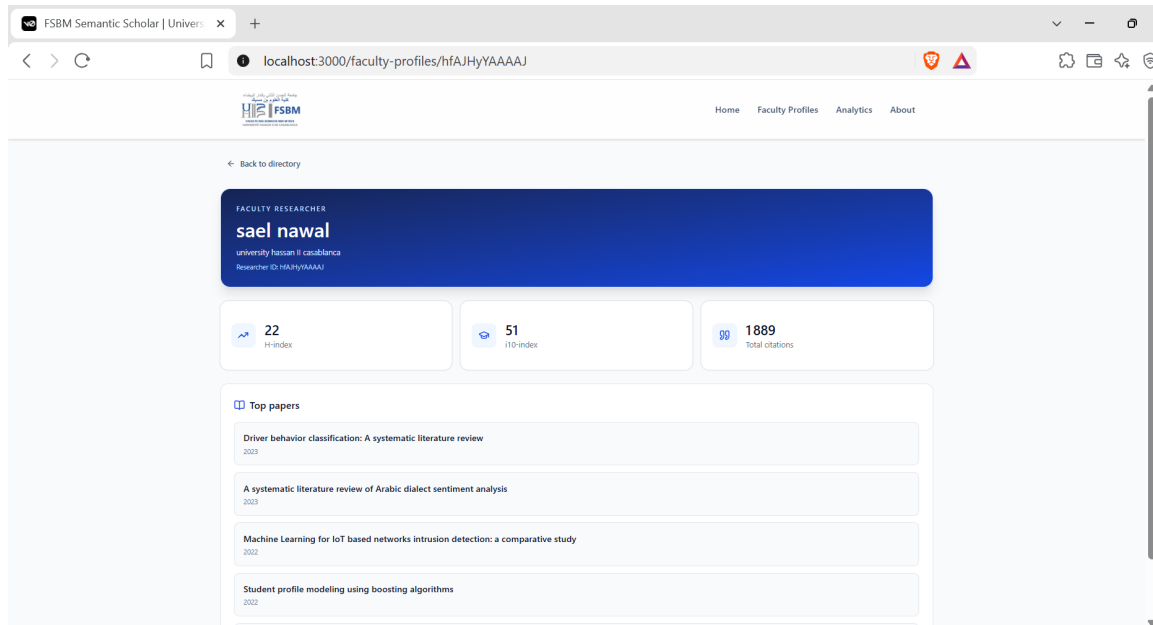


Figure 3.3: Individual researcher profile view displaying historical metrics and indexed top papers.

As depicted in Figure 3.3, the detailed view presents:

- **Header Banner:** Researcher name, institution, Google Scholar ID, and verified research laboratory.
- **Global Metrics Summary:** Responsive summary cards reflecting the researcher's overall academic footprint.
- **Top Papers List:** Chronologically ordered scientific publications, indexed abstracts, and direct reference links.

---

# Chapter 4

## Conclusion & Perspectives

---

### 4.1 Summary of Achievements

This project successfully demonstrates the integration of robust data engineering with modern AI search techniques. By automating the extraction of academic profiles, bypassing complex publisher paywalls using headless browsing, and indexing the data semantically via ChromaDB, we have created a highly scalable platform for exploring the scientific output of FSBM.

### 4.2 Future Work

Future iterations of this platform could include:

- **Automated Summarization:** Integrating Large Language Models (LLMs) to generate localized summaries of complex research papers.
- **Graph Visualization:** Implementing network graphs to visualize collaboration and citation ecosystems among university researchers.
- **Continuous Integration:** Deploying the scraping orchestrator as a scheduled CRON job on a cloud server to keep the database synchronized with new publications automatically.